Desenvolvimento de Apps para Mobile

Fabrício Tonetto Londero

Aula 02





Android

- O Android, sistema operacional móvel **amplamente utilizado**, é conhecido por sua **flexibilidade e adaptação** a uma variedade de dispositivos. **Desenvolvido pela Google**, ele **é baseado no Kernel do Linux** e oferece uma plataforma para criar aplicativos móveis inovadores e interativos (Lecheta, 2010).
- Ele é composto por várias camadas, cada uma desempenhando um papel crucial no funcionamento geral do sistema. Essas camadas, quando combinadas, formam a estrutura que permite a execução de aplicativos e a interação com o hardware do dispositivo.

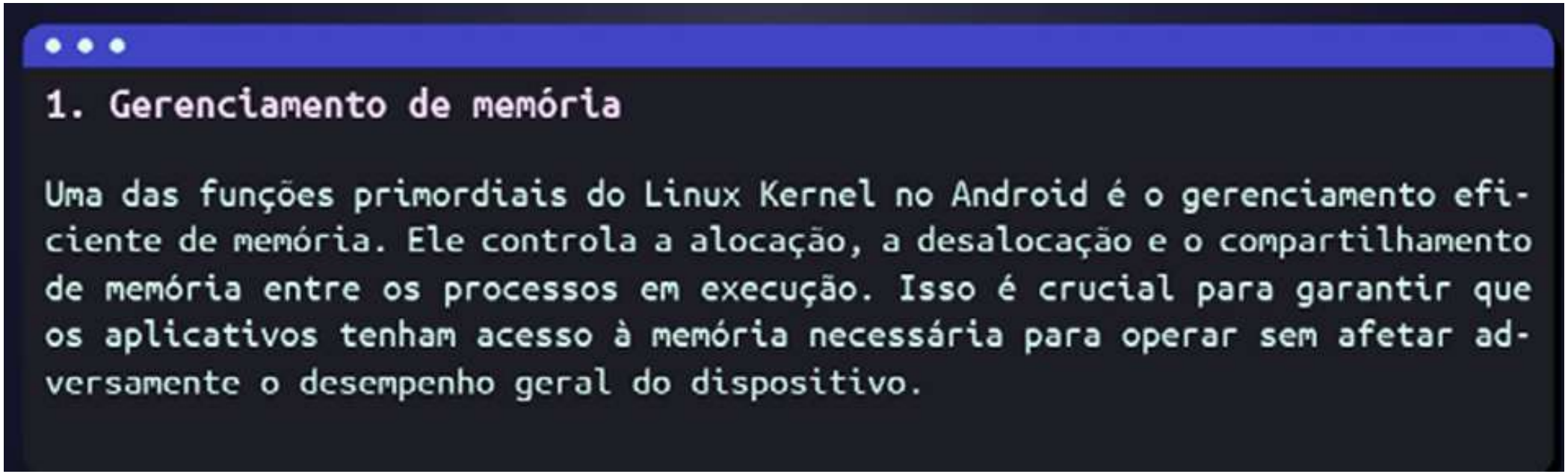


Linux Kernel: base do sistema Android

- O Linux Kernel é a fundação do Android. Ele **fornece os recursos fundamentais do sistema operacional**, como gerenciamento de memória, segurança, pilha de rede e drivers de dispositivo. Sua presença permite a comunicação entre o hardware do dispositivo e as camadas superiores do sistema (Lecheta, 2010).

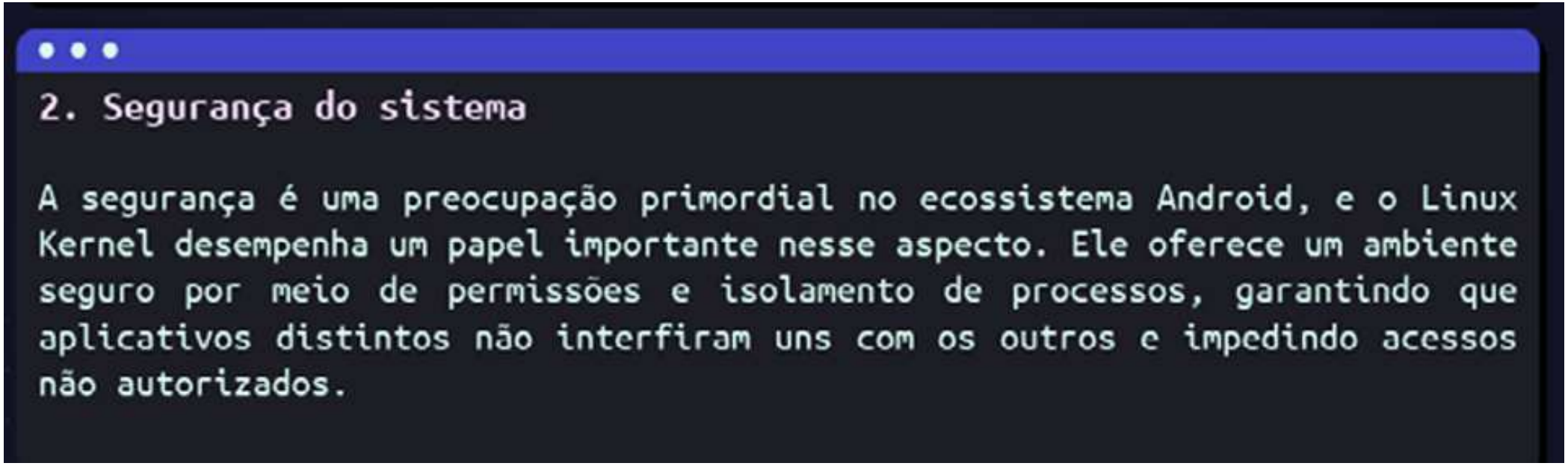
Princípios

Fonte: material do professor
Marcelo Henrique dos Santos



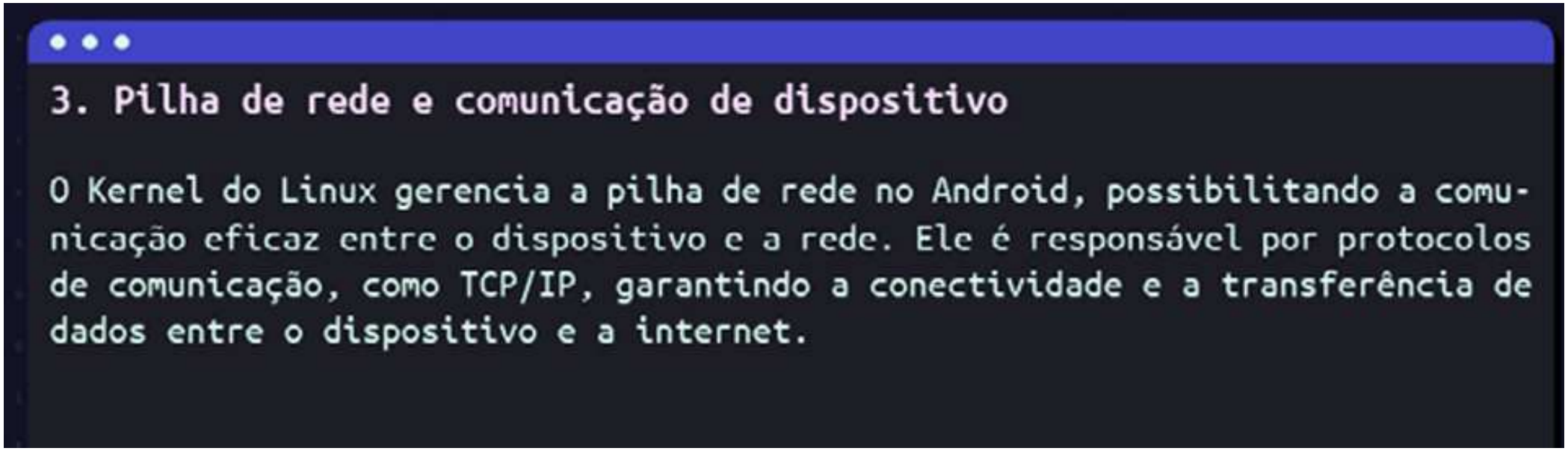
Princípios

Fonte: material do professor Marcelo Henrique dos Santos



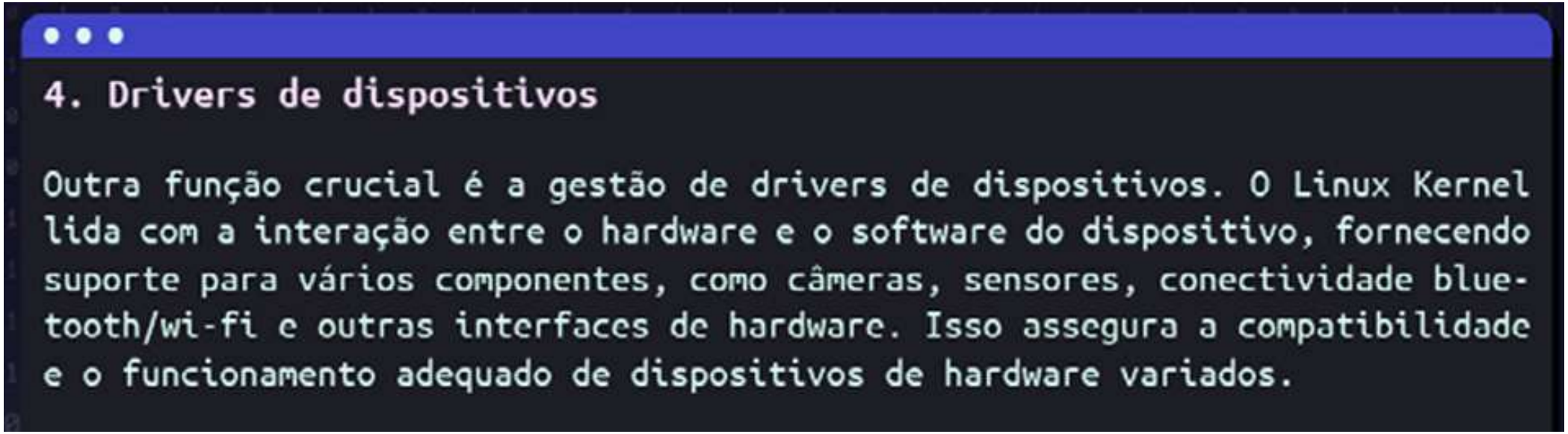
Princípios

Fonte: material do professor
Marcelo Henrique dos Santos



Princípios

Fonte: material do professor
Marcelo Henrique dos Santos



Android Runtime (ART)

- O Android Runtime (ART) é o componente responsável pela execução dos aplicativos Android no dispositivo. Uma de suas características fundamentais é **a conversão do código Java em código de máquina no momento da instalação do aplicativo**.
- Ao contrário do antigo Dalvik, o ART utiliza o processo de compilação AOT para converter o bytecode do aplicativo (gerado a partir do código-fonte em Java) em código de máquina específico do dispositivo no momento da instalação. Isso resulta em uma **execução mais rápida durante a inicialização do aplicativo**, uma vez que o código já está compilado. De acordo com Deitel et al. (2012**), a compilação antecipada permite que o sistema reserve recursos de maneira mais eficiente, reduza a sobrecarga do dispositivo e garanta maior eficiência de bateria**.

Dalvik: Era o antigo ambiente de execução utilizado pelo Android antes da versão 5.0 (Lollipop). Ele executava arquivos .dex (Dalvik Executable).

Bytecode: É o código intermediário gerado quando o código-fonte Java é compilado. Esse bytecode é executado pela máquina virtual do Android (ART ou Dalvik).

Android Runtime (ART)

- Em resumo, a utilização do Android Runtime (ART) no ambiente Android aprimora significativamente a experiência do usuário. Essa tecnologia é uma das razões pelas quais os dispositivos Android oferecem um desempenho ágil e eficiente em uma ampla gama de aplicativos disponíveis na plataforma



Camada de Aplicativos

- A camada de aplicativos é a **parte superior da estrutura** do sistema Android.
- Nela **residem os aplicativos do sistema**, essenciais para o funcionamento do dispositivo, bem como **os aplicativos desenvolvidos por terceiros, disponíveis na Google Play Store ou instalados manualmente**.

Google Play Store: É a loja oficial de aplicativos para dispositivos Android. Nela, os usuários podem baixar aplicativos, jogos, músicas, filmes, livros e outros conteúdos para seus dispositivos.

Princípios da Camada de Aplicativos

1. Execução dos aplicativos

Cada aplicativo Android é executado em seu próprio processo, isolado dos demais aplicativos em execução no dispositivo. Isso é essencial para garantir a segurança e a estabilidade do sistema. Cada processo do aplicativo possui sua própria instância da máquina virtual do Android (Dalvik ou ART), permitindo que o aplicativo execute seu código de maneira independente e segura.

Princípios da Camada de Aplicativos

2. Segurança e isolamento de processos

O isolamento de processos é uma medida crucial para a segurança do sistema operacional Android. Cada aplicativo é executado em um ambiente sandbox, o que significa que ele tem acesso limitado aos recursos do sistema e aos dados de outros aplicativos. Isso impede que um aplicativo comprometido cause danos ou acesse informações confidenciais de outros aplicativos ou do sistema.

Princípios da Camada de Aplicativos

3. Aplicativos do sistema e de terceiros

Os aplicativos do sistema, fornecidos pelo fabricante ou pela operadora, incluem componentes essenciais para o funcionamento do dispositivo, como o launcher, o gerenciador de pacotes e o teclado. Já os aplicativos de terceiros são desenvolvidos por diferentes fornecedores e oferecem uma variedade de funcionalidades, desde redes sociais até aplicativos de produtividade, jogos e muito mais.

Framework

- O framework do Android **fornece um conjunto abrangente de recursos para os desenvolvedores**. Ele oferece acesso a recursos do sistema, como a Interface do Usuário (UI), os recursos de mídia, a conectividade de rede e hardware para criar funcionalidades e interações entre o usuário e o aplicativo.
- Compreender sua arquitetura e suas camadas é importante para criar aplicativos eficientes e inovadores para essa plataforma. Com todos esses dados em mente, chegou o momento de entender o ambiente de desenvolvimento Android.

Framework do Android: É um conjunto de APIs e bibliotecas fornecidas pelo Android para permitir o desenvolvimento de aplicativos. Oferece funcionalidades para interação com o sistema operacional, hardware, serviços do Google e outros recursos.

Interface do Usuário (UI): Refere-se à parte do aplicativo ou sistema que permite a interação entre o usuário e o dispositivo, como telas, botões, menus e elementos interativos.

Ambiente de desenvolvimento Android

- O ambiente de desenvolvimento Android desempenha um papel essencial na construção de aplicativos móveis, fornecendo a base para a criação de soluções inovadoras e eficazes. Nesse contexto, **o Android Studio**, consagrado como a **IDE (*Integrated Development Environment*) oficial do Google para o desenvolvimento Android**; representa não apenas uma ferramenta, mas sim **uma plataforma abrangente que reúne uma série de recursos**.

Android Studio: É a IDE oficial para desenvolvimento de aplicativos Android. Oferece ferramentas para criação de aplicativos, depuração, gerenciamento de recursos e emulação de dispositivos.

IDE: Ambiente integrado de desenvolvimento com ferramentas para escrever, depurar e testar programas. O Android Studio é uma IDE oficial do Google para desenvolvimento de aplicativos Android.

Instalação e configuração do Android Studio

- A instalação e configuração do Android Studio representam os primeiros passos para os desenvolvedores que desejam criar aplicativos móveis na plataforma Android. Esse processo é fundamental para estabelecer o ambiente de desenvolvimento e acessar as ferramentas e recursos disponíveis na IDE oficial do Google para o desenvolvimento Android.



Processo de Instalação e Configuração

Passo 1 - Download

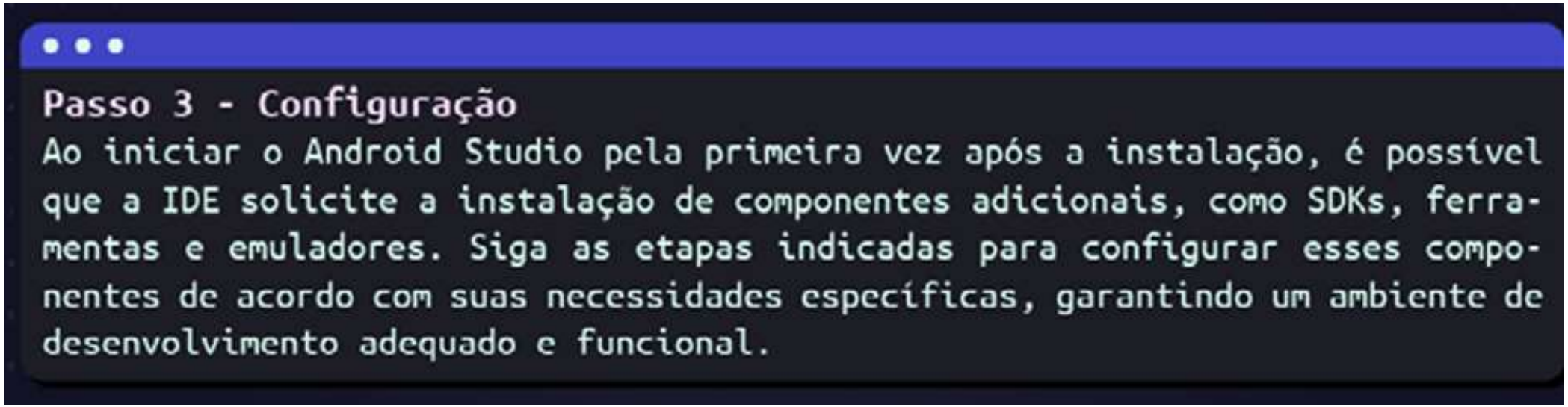
Acessar o site oficial *(https://developer.android.com/studio)* e realizar o download da versão compatível com o seu sistema operacional. Essa é a porta de entrada para ter acesso à IDE e suas funcionalidades.

Processo de Instalação e Configuração

Passo 2 - Instalação

Após o download, execute o instalador do Android Studio e siga as instruções fornecidas para concluir o processo de instalação. Esse passo é crucial para ter o ambiente de desenvolvimento pronto para uso.

Processo de Instalação e Configuração



Requisitos

Requisitos do sistema para Windows:

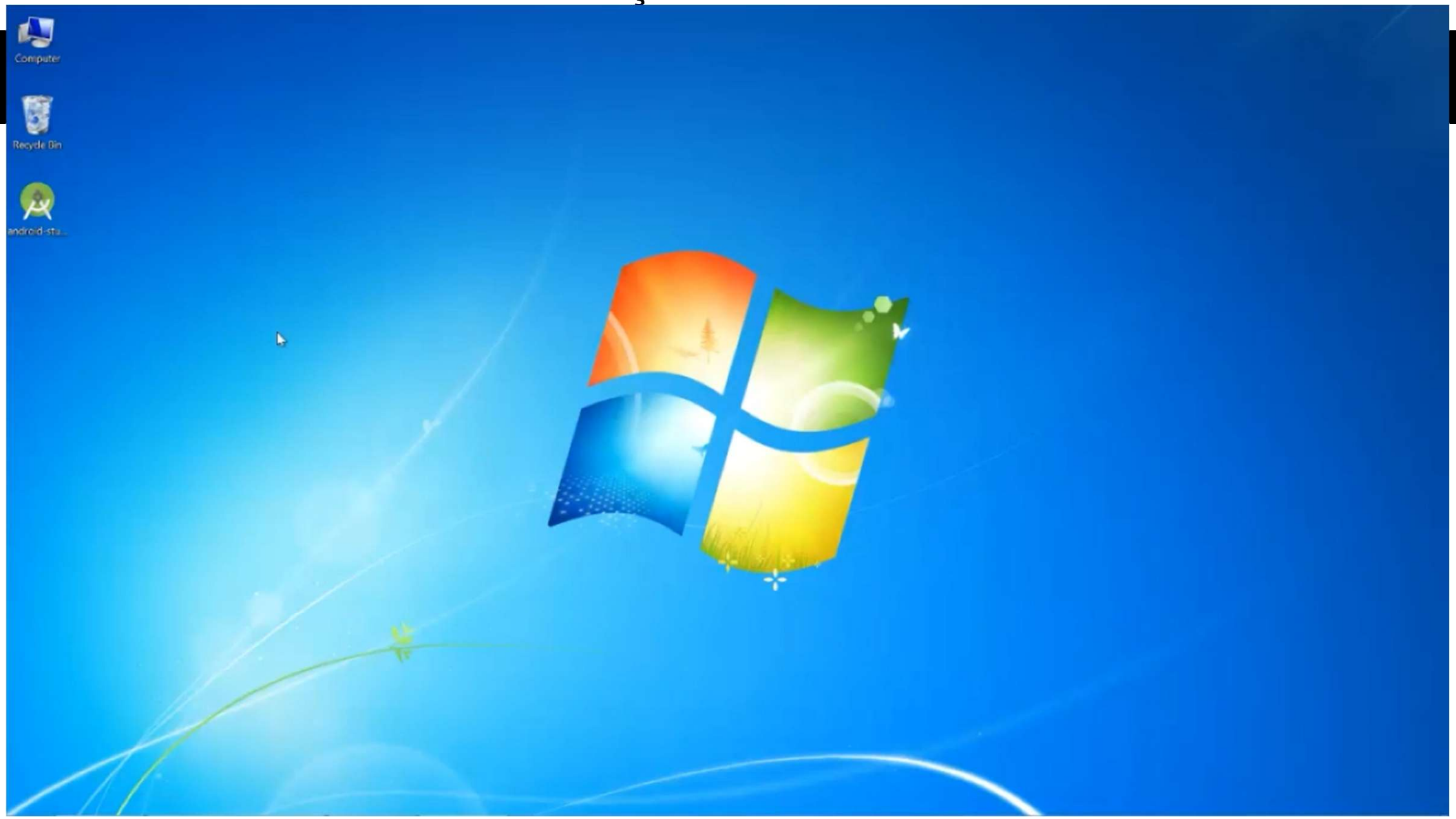
Requisito	Mínimo	Recomendado
SO	Microsoft Windows 8 de 64 bits	Versão de 64 bits mais recente do Windows
RAM	8 GB de RAM	16 GB de RAM ou mais
CPU	Arquitetura de CPU x86_64, Intel Core de segunda geração ou mais recente ou CPU AMD com suporte ao Hypervisor Framework do Windows	Processador Intel Core mais recente
Espaço em disco	8 GB (ambiente de desenvolvimento integrado, Android Emulator e SDK do Android)	Unidade de estado sólido (SSD) com 16 GB ou mais
Resolução da tela	1280 x 800	1920 x 1080

Instalação no Windows

Para instalar o Android Studio no Windows, siga estas etapas:

- Se você fez o download de um arquivo `.exe` (recomendado), clique nele duas vezes para iniciá-lo.
- Se você fez o download de um arquivo `.zip`:
    1. Descompacte o `.zip`.
    2. Copie a pasta **android-studio** para a pasta **Arquivos de Programas**.
    3. Abra a pasta **android-studio > bin**.
    4. Inicie `studio64.exe` (para máquinas de 64 bits) ou `studio.exe` (para máquinas de 32 bits).
    5. Siga o **assistente de configuração** no Android Studio e instale todos os pacotes do SDK recomendados.

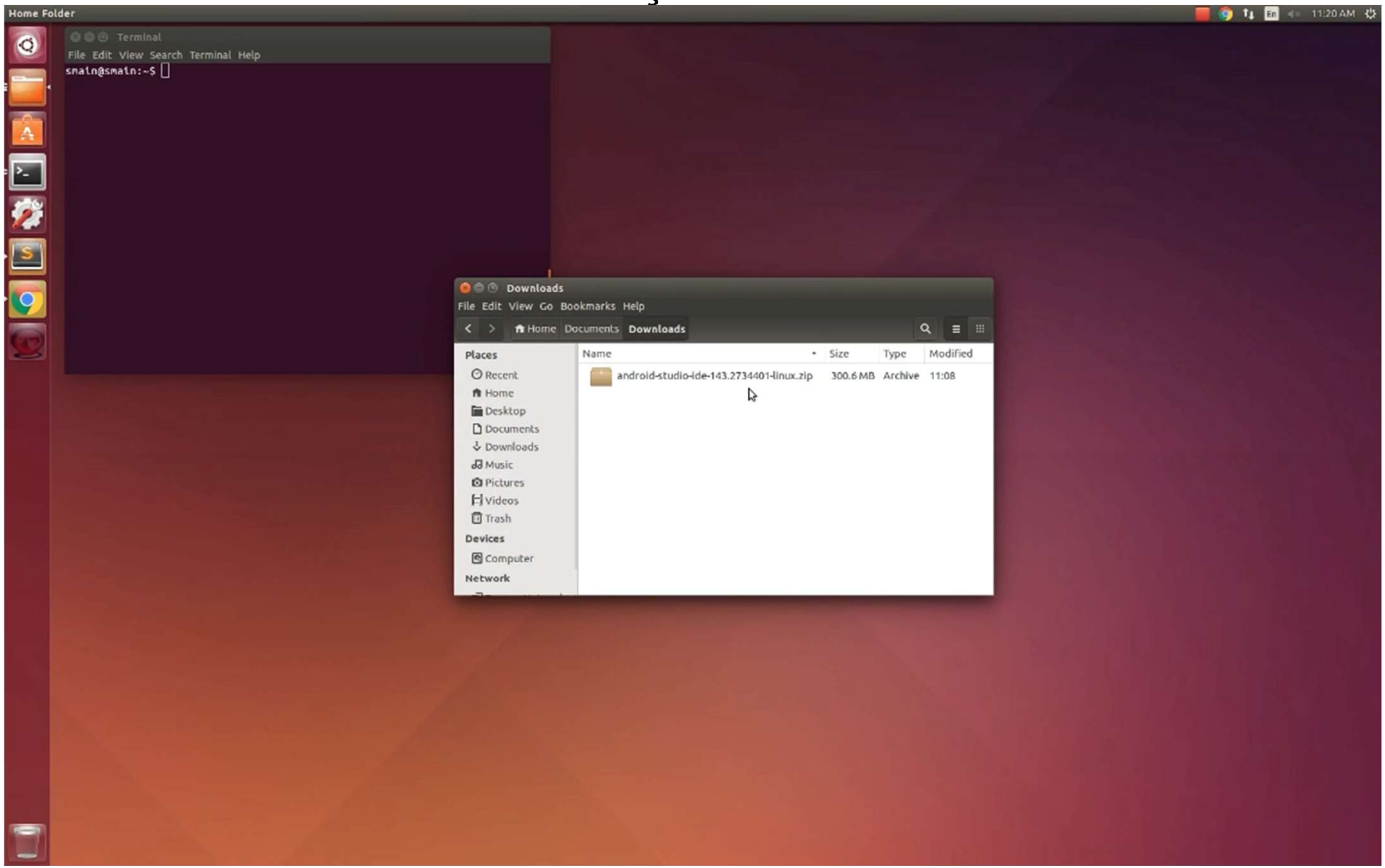
Instalação no Windows



Instalação no Mac



Instalação no Linux

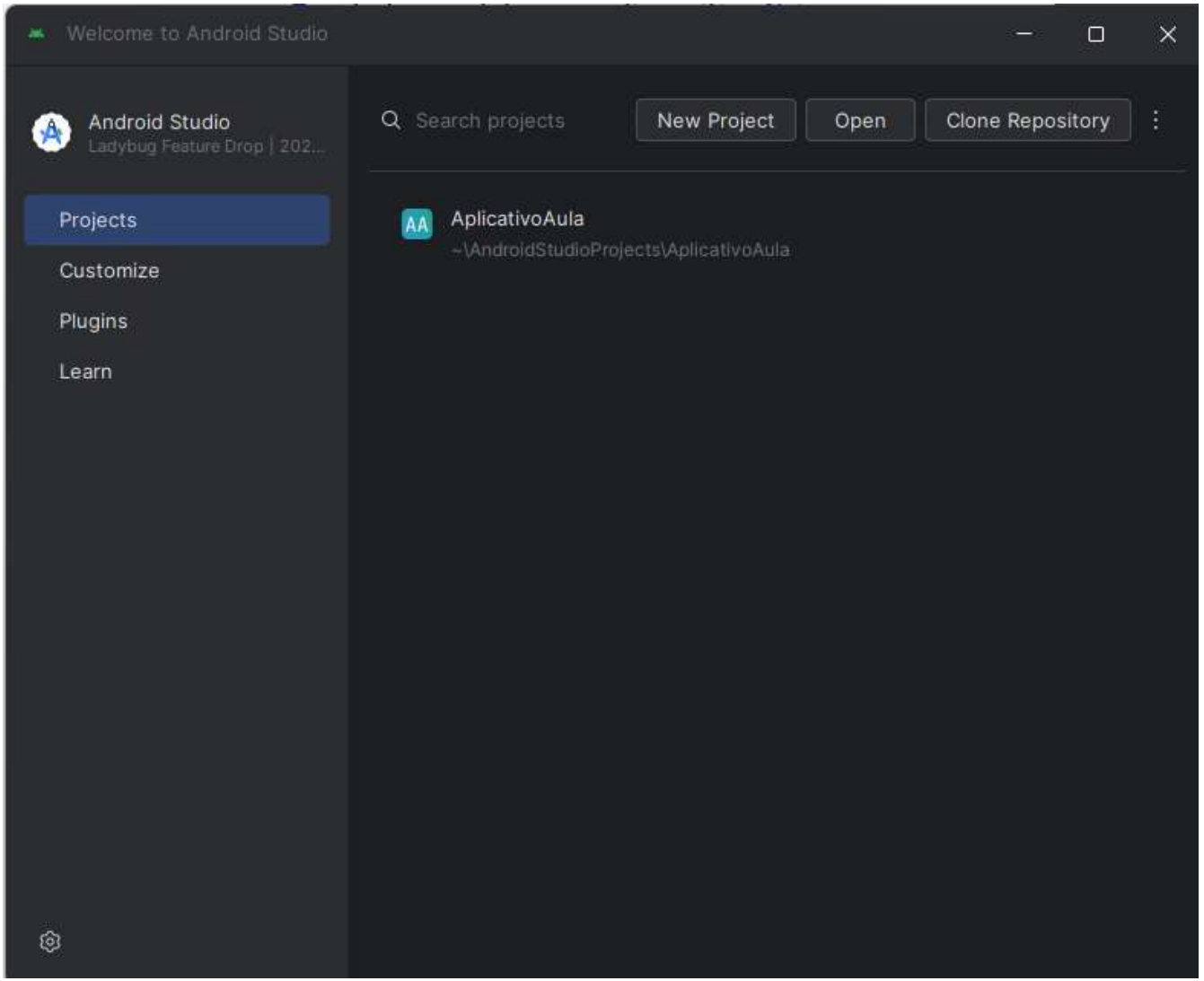


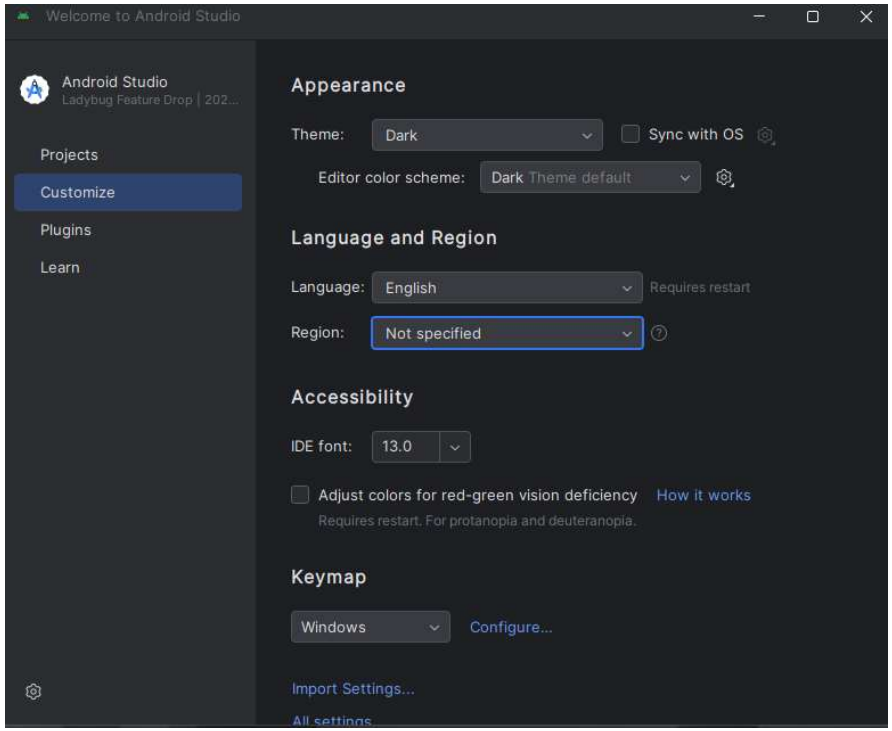
Primeiro Projeto

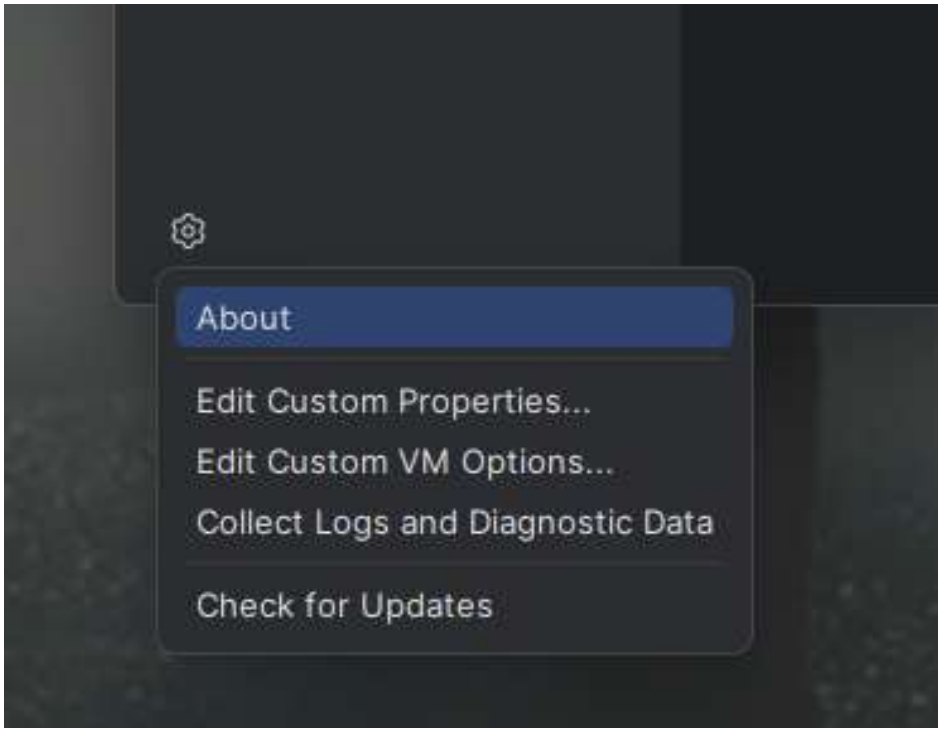
- Abrindo o Android Studio



Primeiro Projeto – Primeira Tela





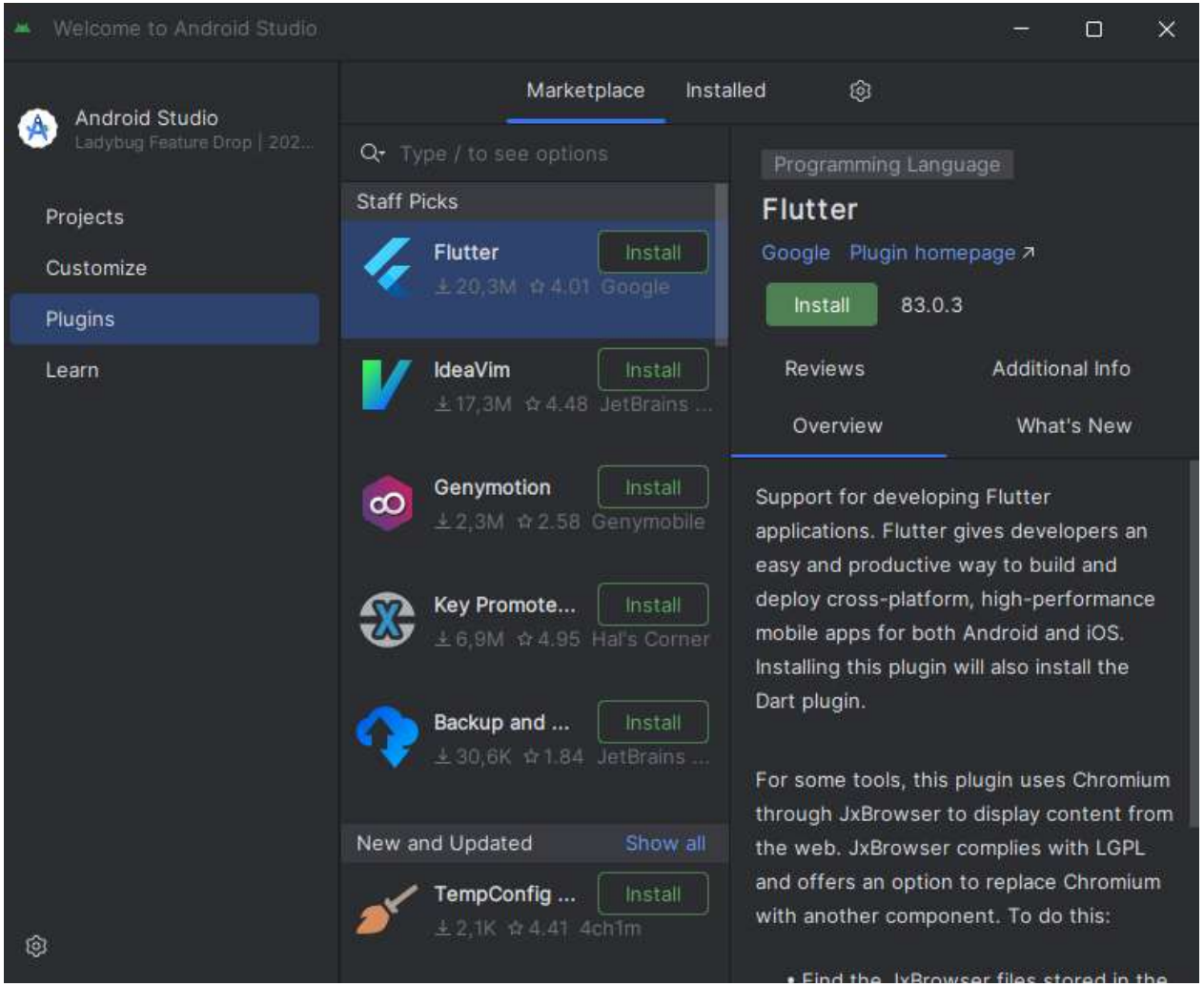


Primeira tela

- Possibilidade de updates (muito importante)
- Customização

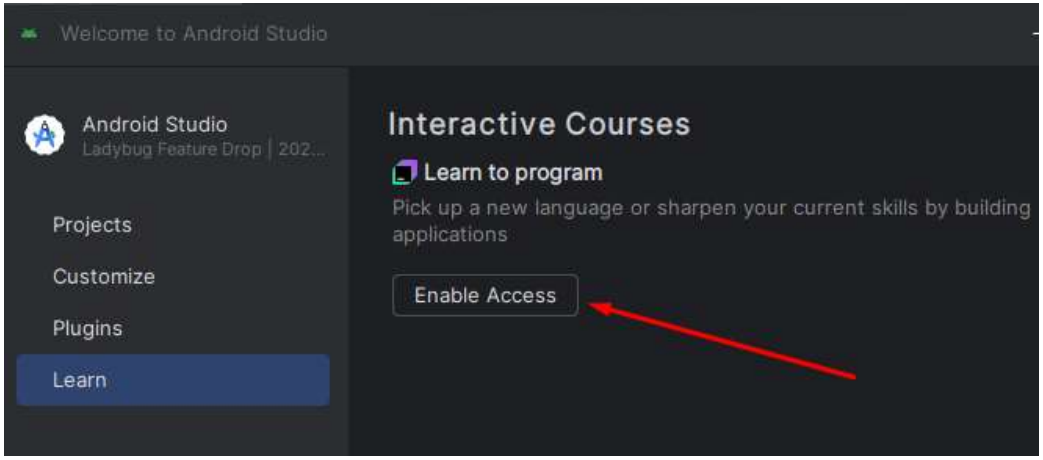
Primeira tela

- Instalação de Plugins

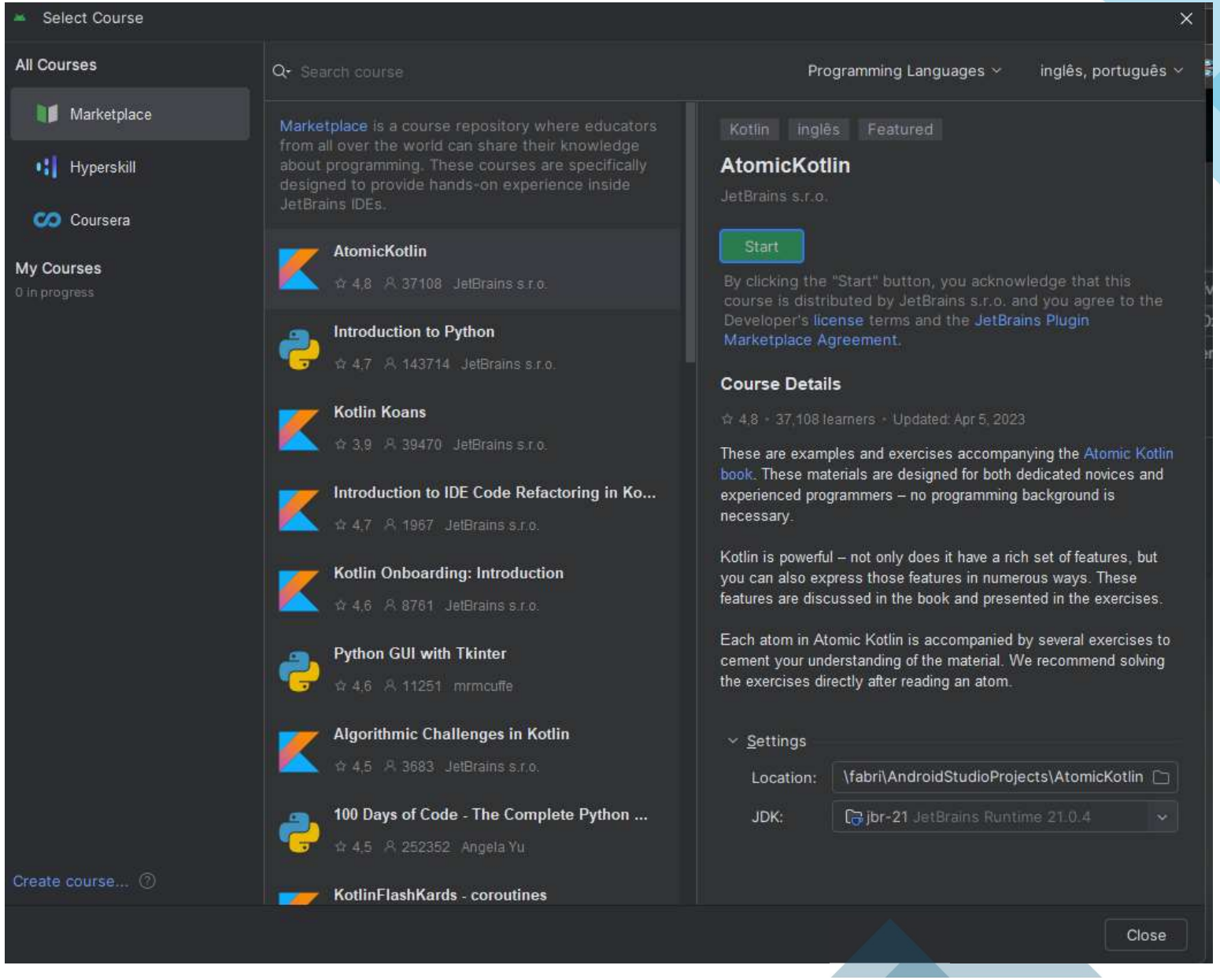


Primeira Tela

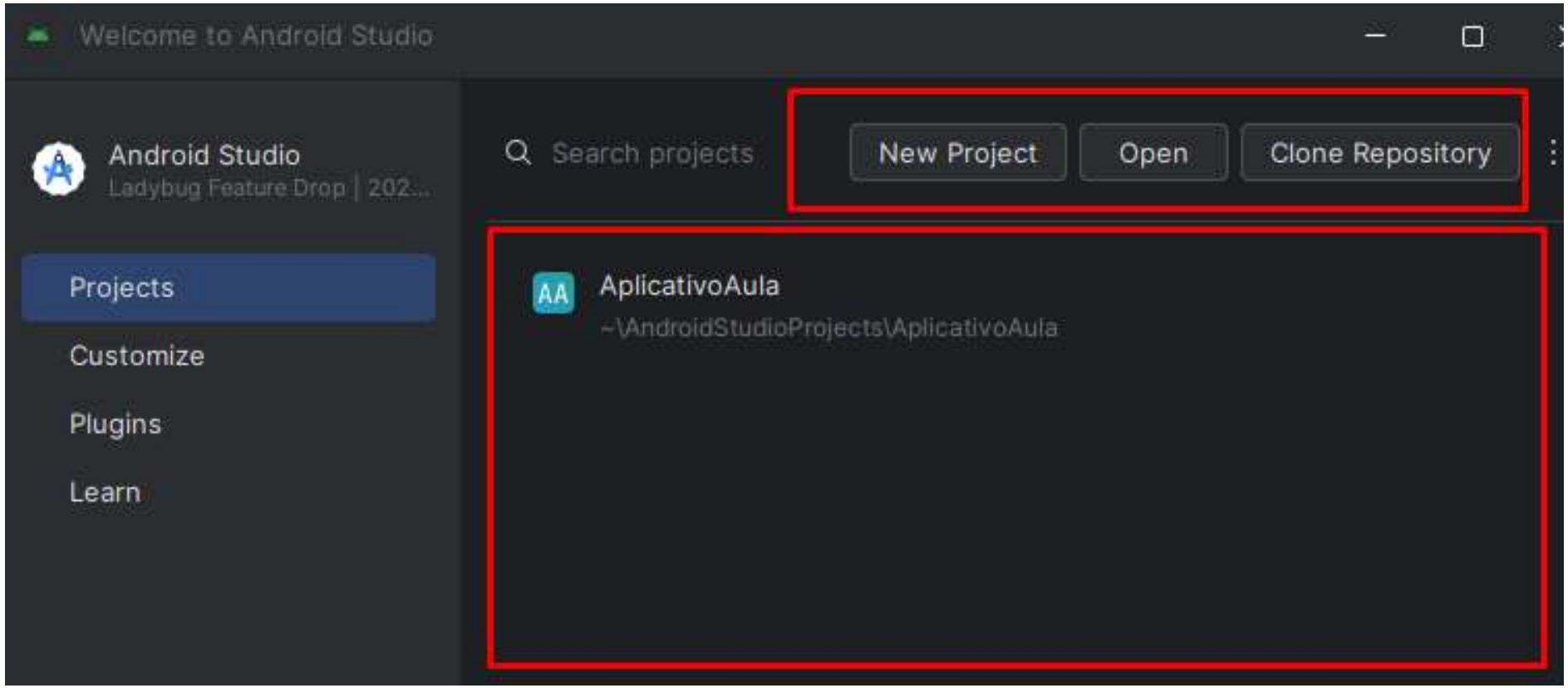
Tela para aprendizado, deve ser habilitada



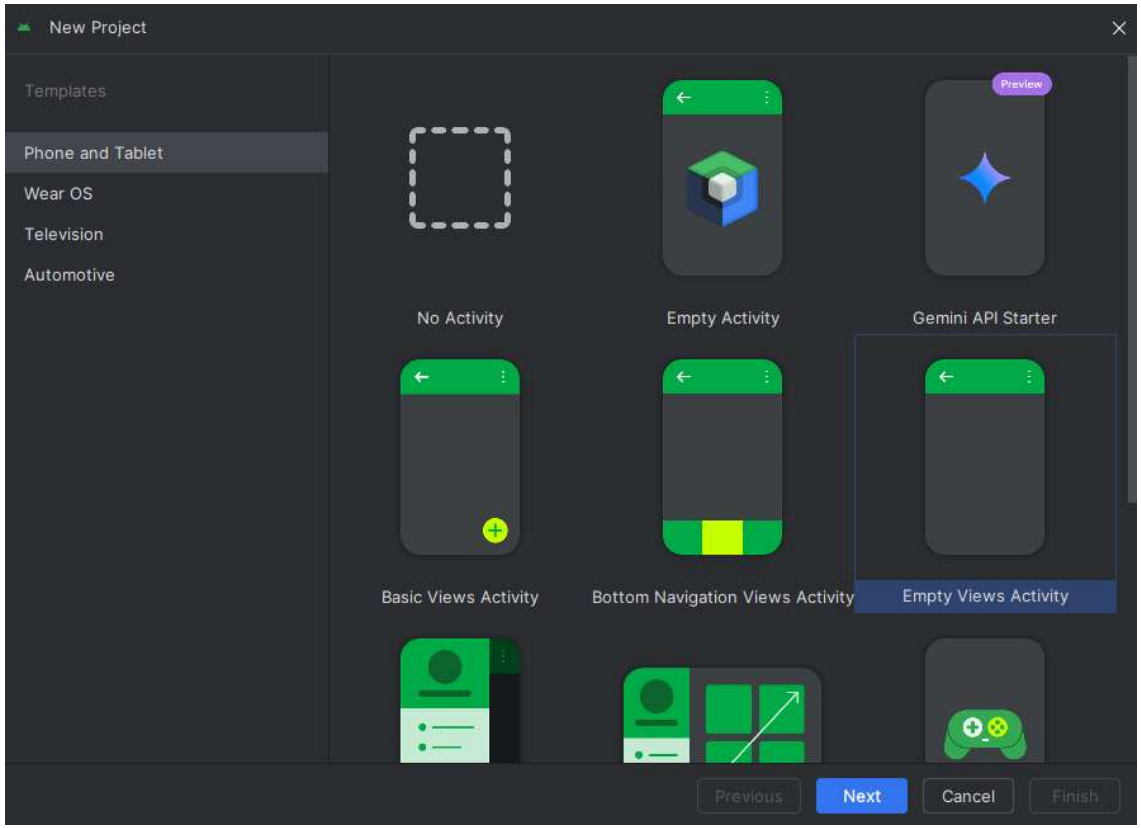



Select Course
All Courses
Marketplace
Hyperskill
Coursera
My Courses
0 in progress
Create course...
Search course
Programming Languages
inglês, português
Marketplace is a course repository where educators from all over the world can share their knowledge about programming. These courses are specifically designed to provide hands-on experience inside JetBrains IDEs.
AtomicKotlin
4,8 37108 JetBrains s.r.o.
Introduction to Python
4,7 143714 JetBrains s.r.o.
Kotlin Koans
3,9 39470 JetBrains s.r.o.
Introduction to IDE Code Refactoring in Ko...
4,7 1967 JetBrains s.r.o.
Kotlin Onboarding: Introduction
4,6 8761 JetBrains s.r.o.
Python GUI with Tkinter
4,6 11251 mrmcuffe
Algorithmic Challenges in Kotlin
4,5 3683 JetBrains s.r.o.
100 Days of Code - The Complete Python ...
4,5 252352 Angela Yu
KotlinFlashKards - coroutines
Kotlin
inglês
Featured
AtomicKotlin
JetBrains s.r.o.
Start
By clicking the "Start" button, you acknowledge that this course is distributed by JetBrains s.r.o. and you agree to the Developer's license terms and the JetBrains Plugin Marketplace Agreement.
Course Details
4,8 • 37,108 learners • Updated: Apr 5, 2023
These are examples and exercises accompanying the Atomic Kotlin book. These materials are designed for both dedicated novices and experienced programmers – no programming background is necessary.
Kotlin is powerful – not only does it have a rich set of features, but you can also express those features in numerous ways. These features are discussed in the book and presented in the exercises.
Each atom in Atomic Kotlin is accompanied by several exercises to cement your understanding of the material. We recommend solving the exercises directly after reading an atom.
Settings
Location:
\fabri\AndroidStudioProjects\AtomicKotlin
JDK:
jbr-21 JetBrains Runtime 21.0.4
Close

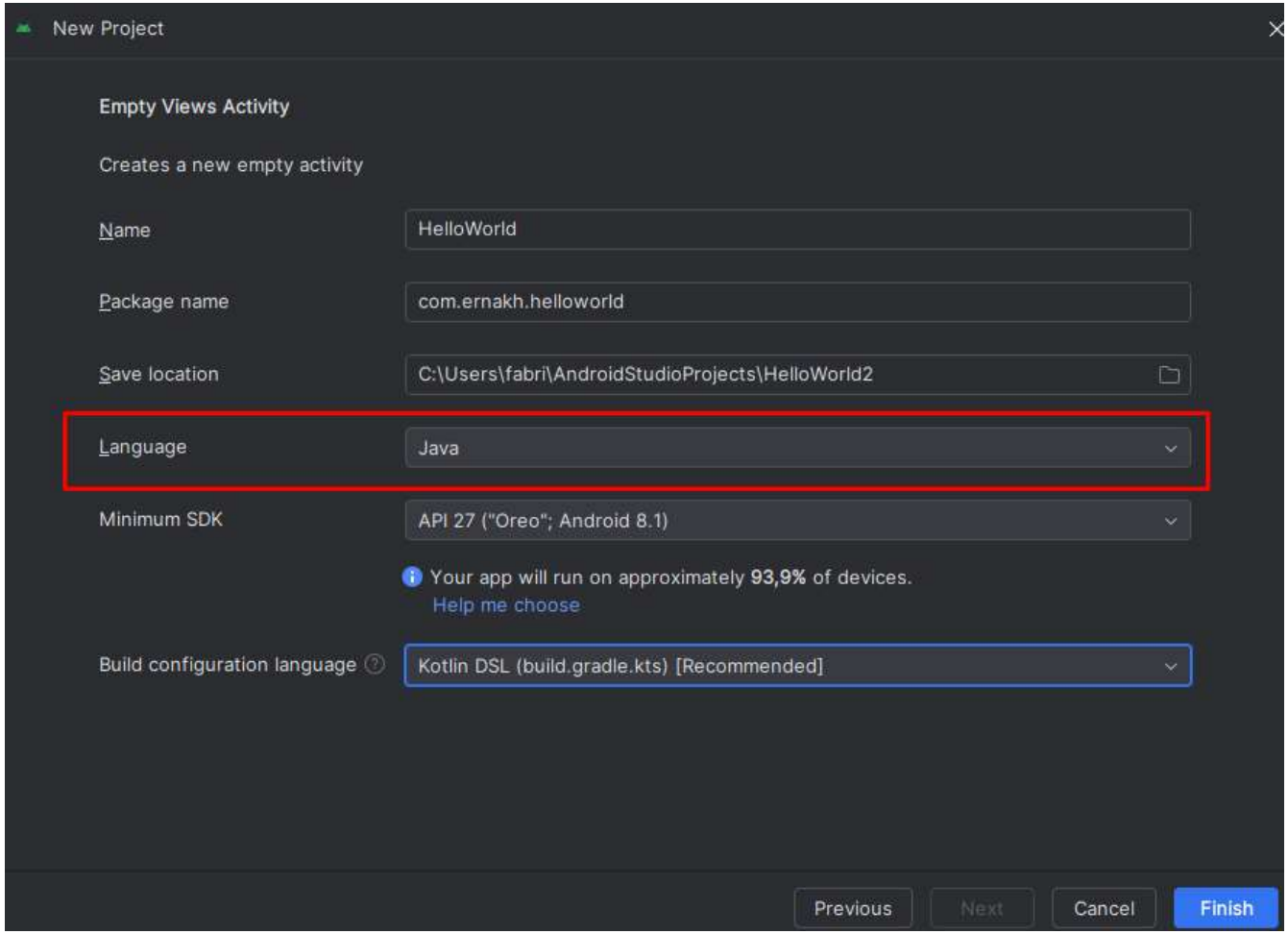
Primeira tela – Gerenciamento de Projetos



Criando um projeto



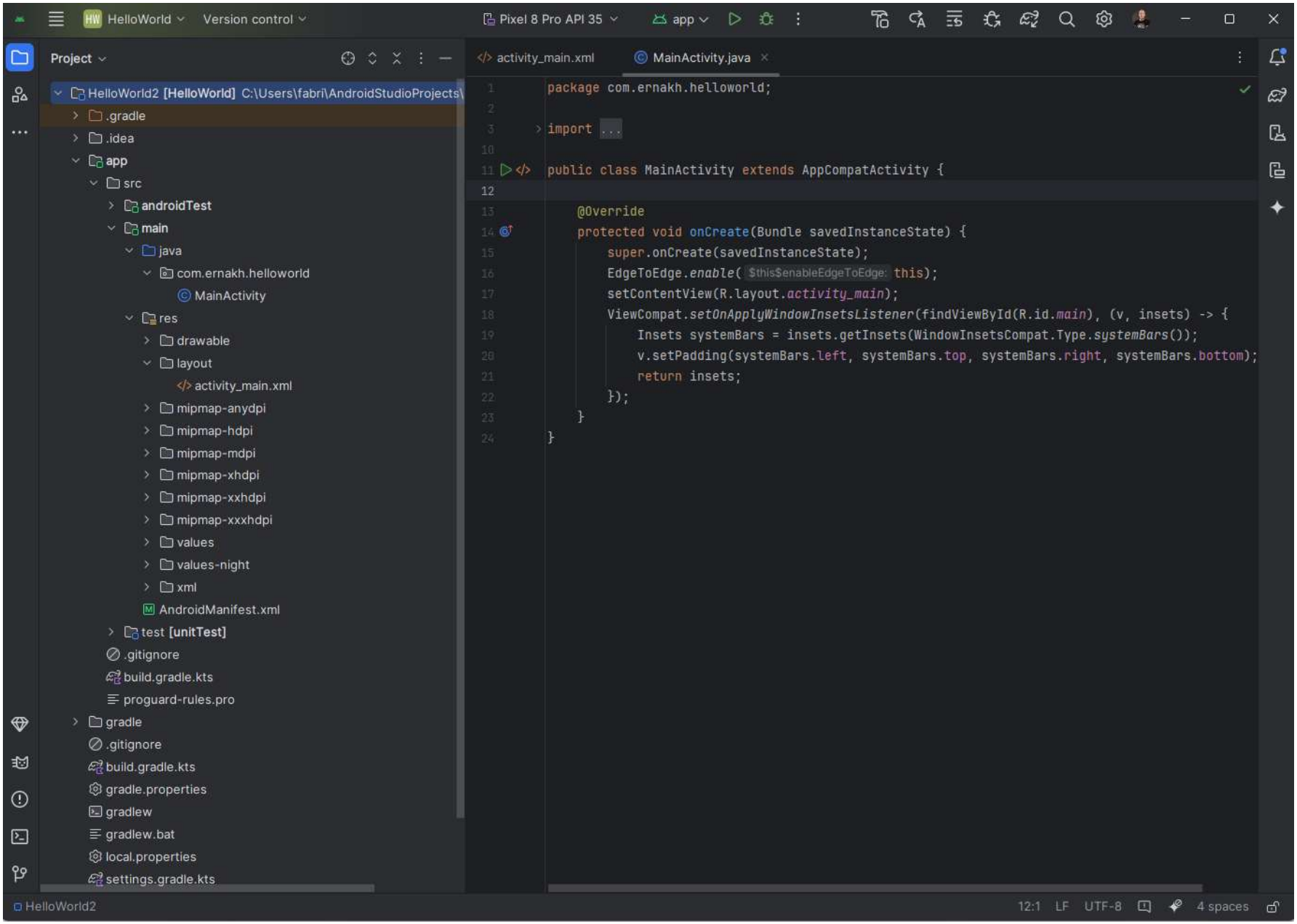
Criando um projeto



Android Studio – Criando o projeto

- Aguarde as dependências serem totalmente carregadas




HelloWorld
Version control
Pixel 8 Pro API 35
app
Project
HelloWorld2 [HelloWorld] C:\Users\fabri\AndroidStudioProjects\
.gradle
.idea
app
src
androidTest
main
java
com.ernakh.helloworld
MainActivity
res
drawable
layout
activity_main.xml
mipmap-anydpi
mipmap-hdpi
mipmap-mdpi
mipmap-xhdpi
mipmap-xxhdpi
mipmap-xxxhdpi
values
values-night
xml
AndroidManifest.xml
test [unitTest]
.gitignore
build.gradle.kts
proguard-rules.pro
gradle
.gitignore
build.gradle.kts
gradle.properties
gradlew
gradlew.bat
local.properties
settings.gradle.kts
activity_main.xml
MainActivity.java
package com.ernakh.helloworld;
import ...
public class MainActivity extends AppCompatActivity {
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
EdgeToEdge.enable($this$enableEdgeToEdge: this);
setContentView(R.layout.activity_main);
ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
return insets;
});
}
}
HelloWorld2
12:1
LF
UTF-8
4 spaces

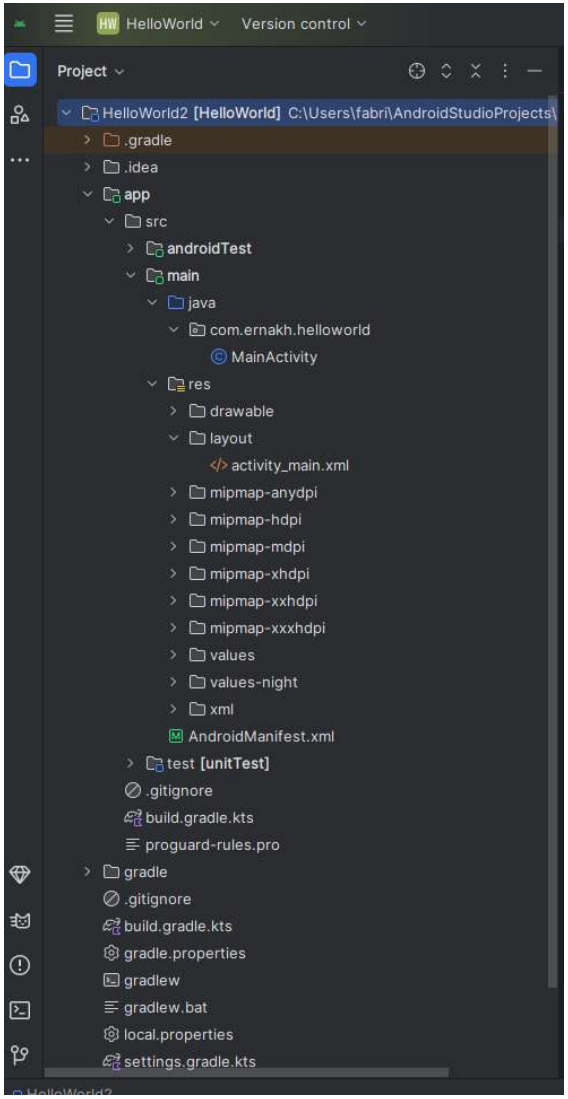


Estrutura de pastas

- Ao gerar um novo projeto no Android Studio, a estrutura de pastas padrão é criada.
- Aqui está um exemplo simplificado:
  - **app/src/main/java:** contém o código-fonte Java do aplicativo;
  - **app/src/main/res:** armazena recursos como layouts, strings, imagens e outros arquivos XML;
  - **app/src/main/AndroidManifest.xml:** arquivo de manifesto que contém informações essenciais sobre o aplicativo.

Arquivos do Projeto

- Projeto com apenas uma Activity (já falaremos sobre isso)
- Pastas e arquivos destaque
  - App/src/main/
    - Java
      - Código fonte Java das Activities e de outra classes
    - Res/layout
      - Arquivos xml, o visual das telas



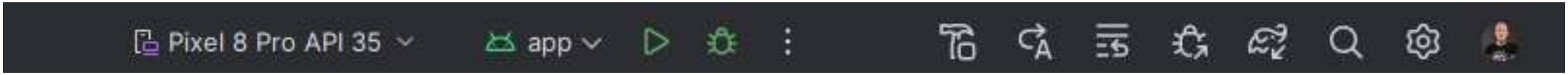
Conceitos Importantes

- Ao desenvolver aplicativos para a plataforma Android, é essencial compreender alguns conceitos básicos que são fundamentais para a construção de interfaces e funcionalidades.
    - A **Activity** é um componente crucial no Android, representando uma única tela com interface de usuário.
    - As **Views** são os componentes visuais dentro de uma Activity, como botões, campos de texto e imagens.
    - A **classe R** é gerada automaticamente pelo Android Studio e contém referências a todos os recursos do projeto, como layouts, imagens e strings.
    - Já o **layout XML** é a maneira de definir a estrutura e a aparência da interface do usuário.



ToolBox

- Atalhos para as funções mais comuns do desenvolvimento Android
  - Rodar a aplicação no Emulador
  - Debug
  - Make
  - Sincronizar as dependências com o Gradle



Editor

Modificar o código fonte e customizar interface gráfica

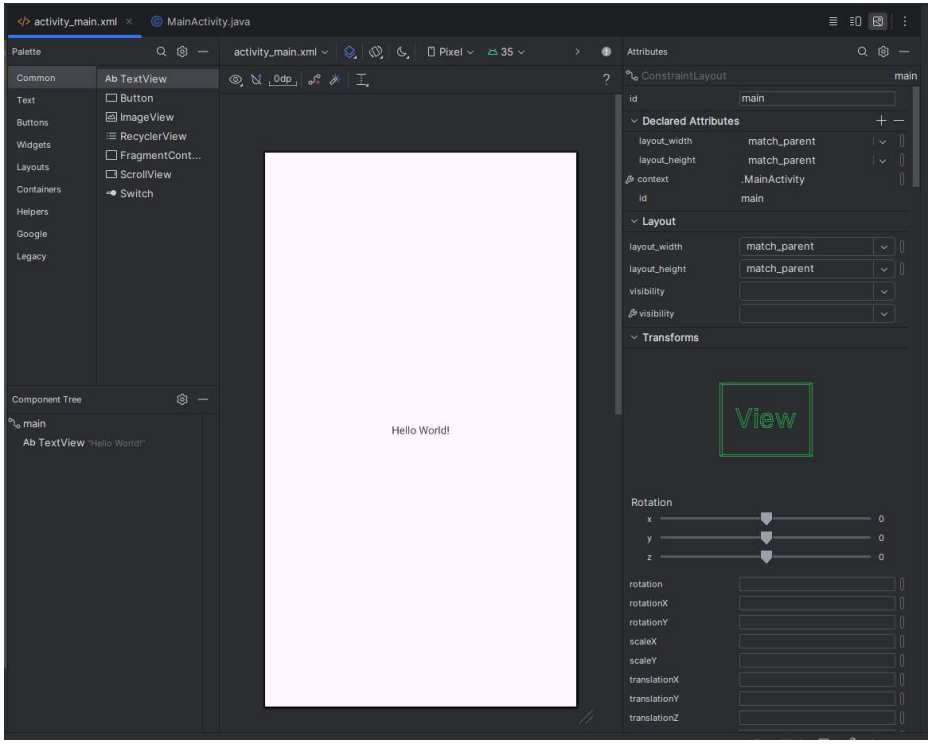
activity_main.xml | MainActivity.java

```java
package com.ernakh.helloworld;

import ...

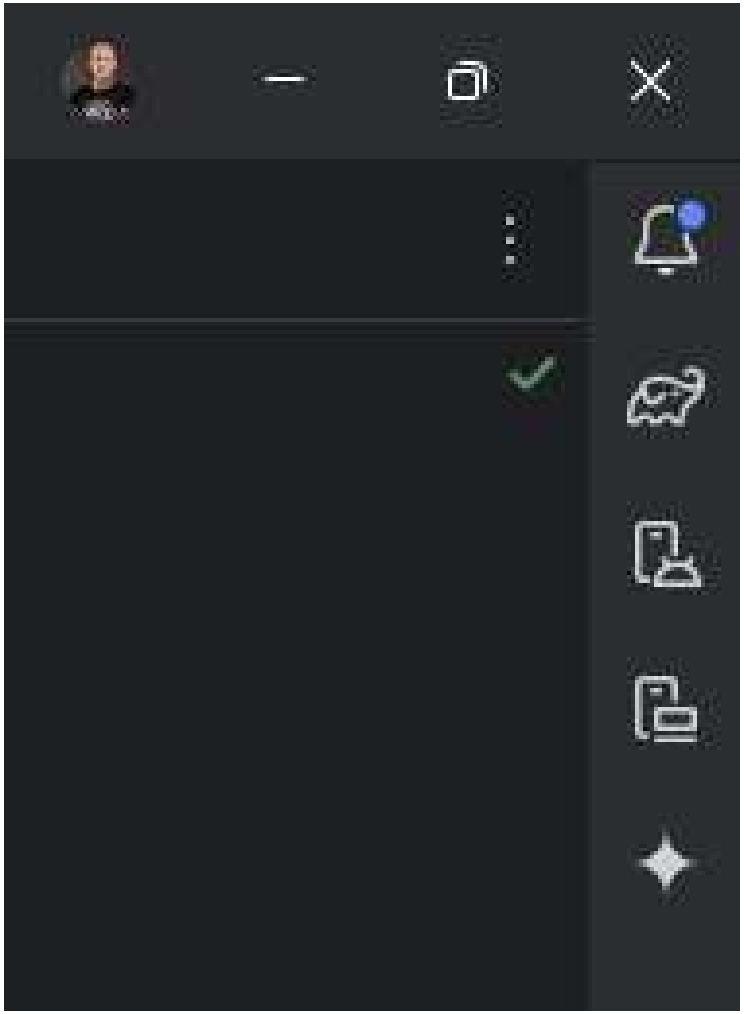
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable( $this$enableEdgeToEdge: this);
        setContentView(R.layout.activity_main);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });
    }
}
```

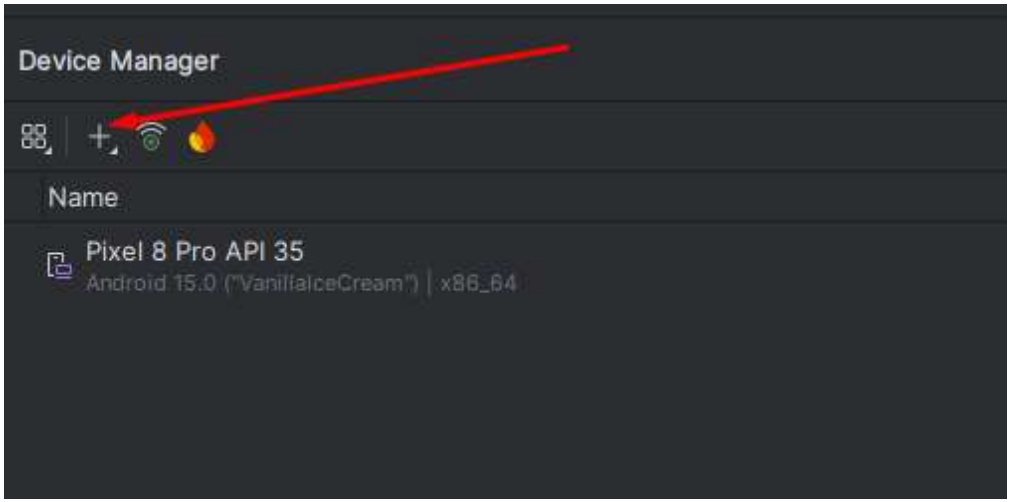


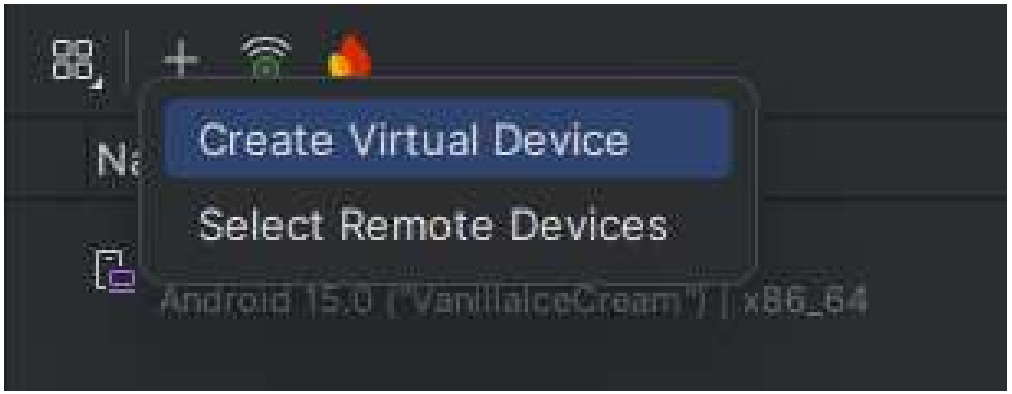
Toolbox

- Destaque para notificações na vertical, atalho para Gradle, Gerenciador de dispositivos e Dispositivos rodando (Emuladores)

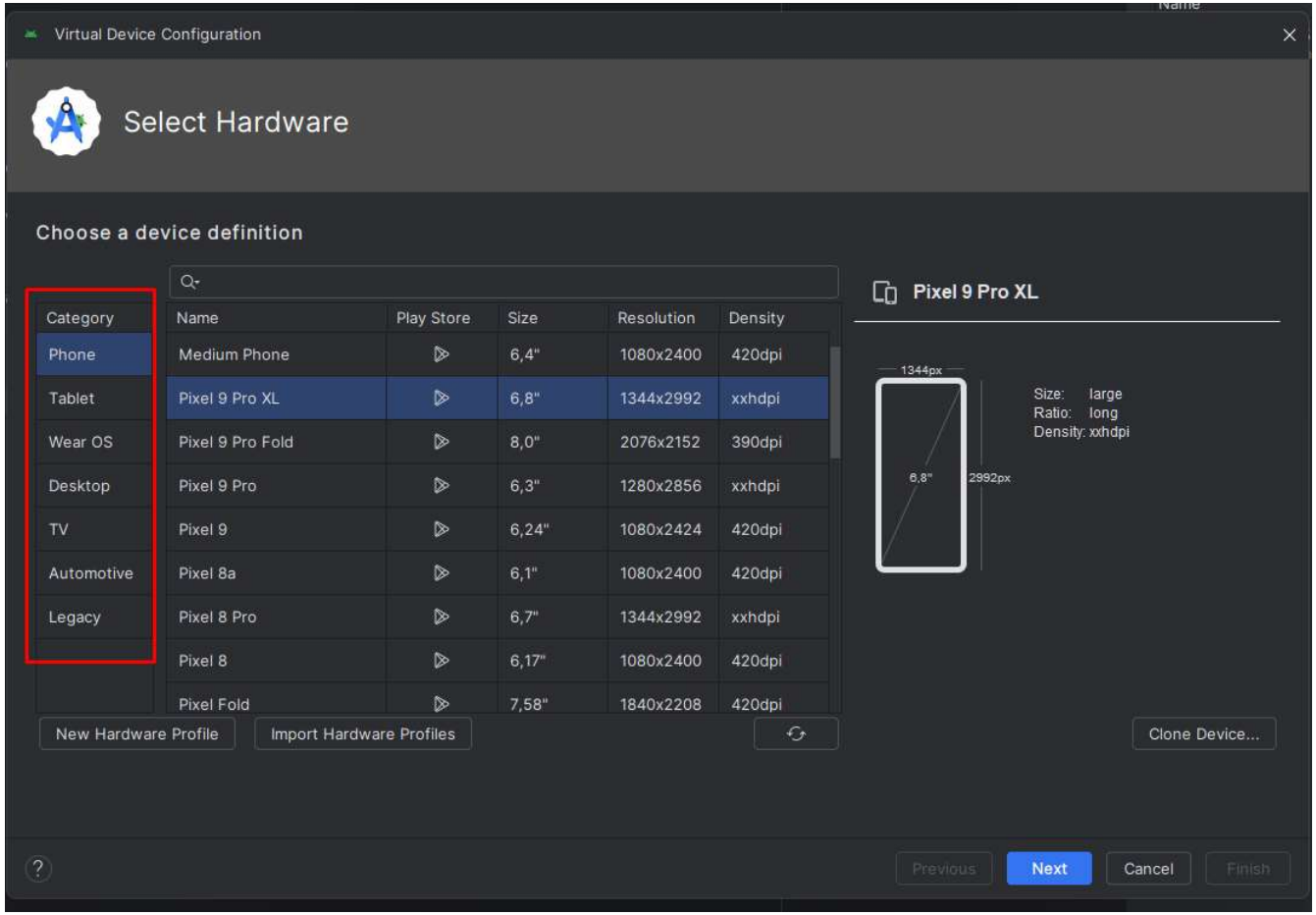


Criando um Dispositivo

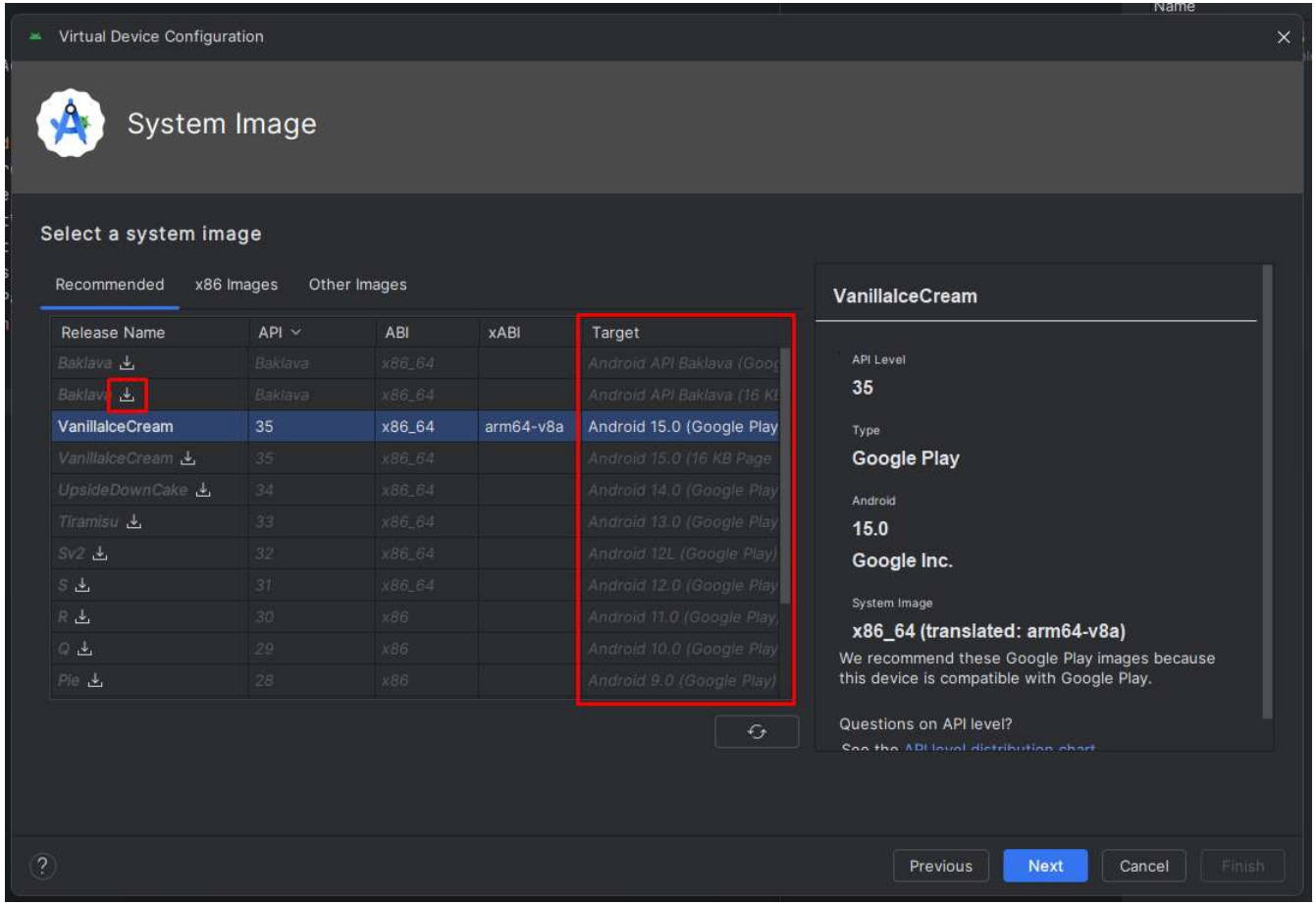




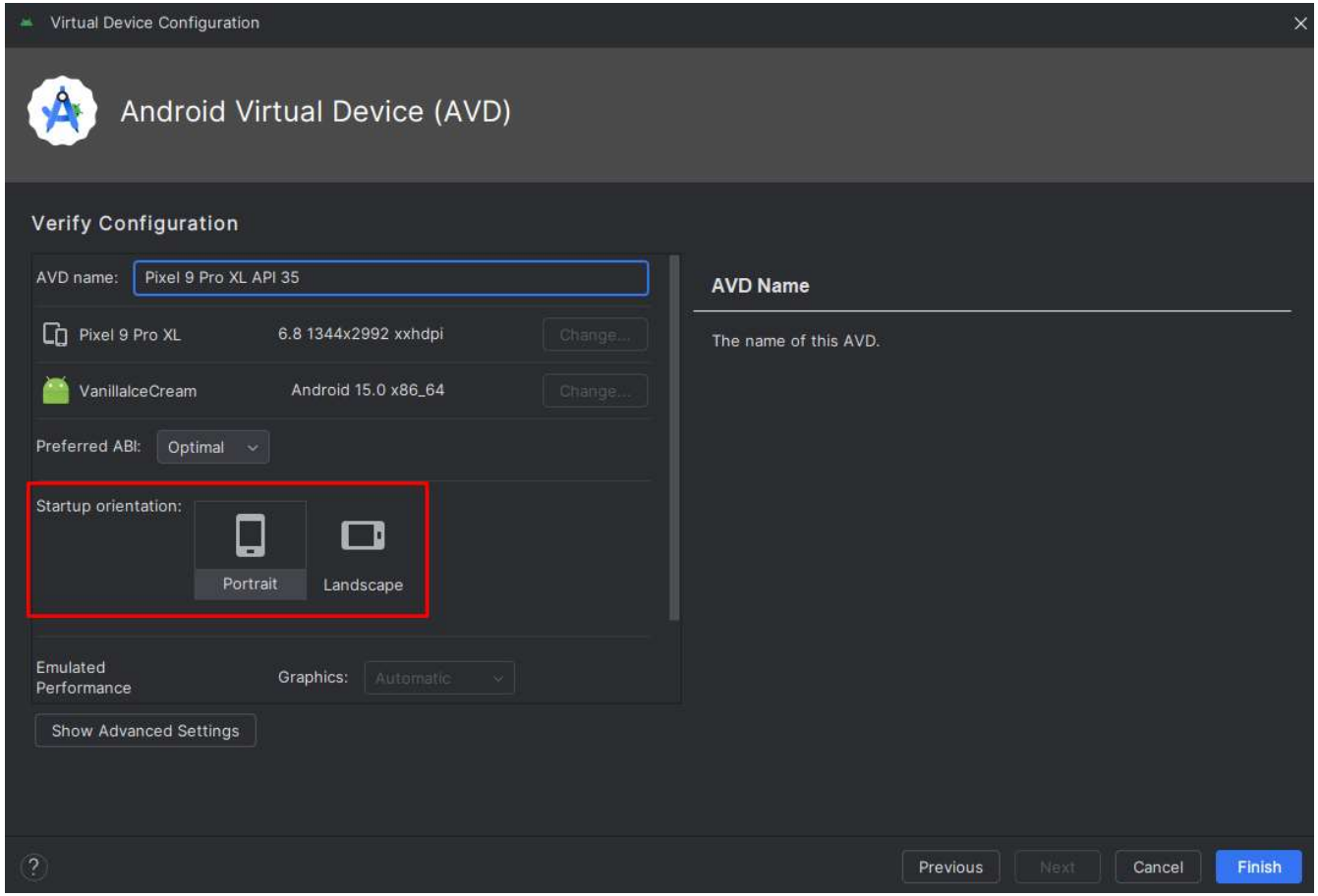
Criando um dispositivo



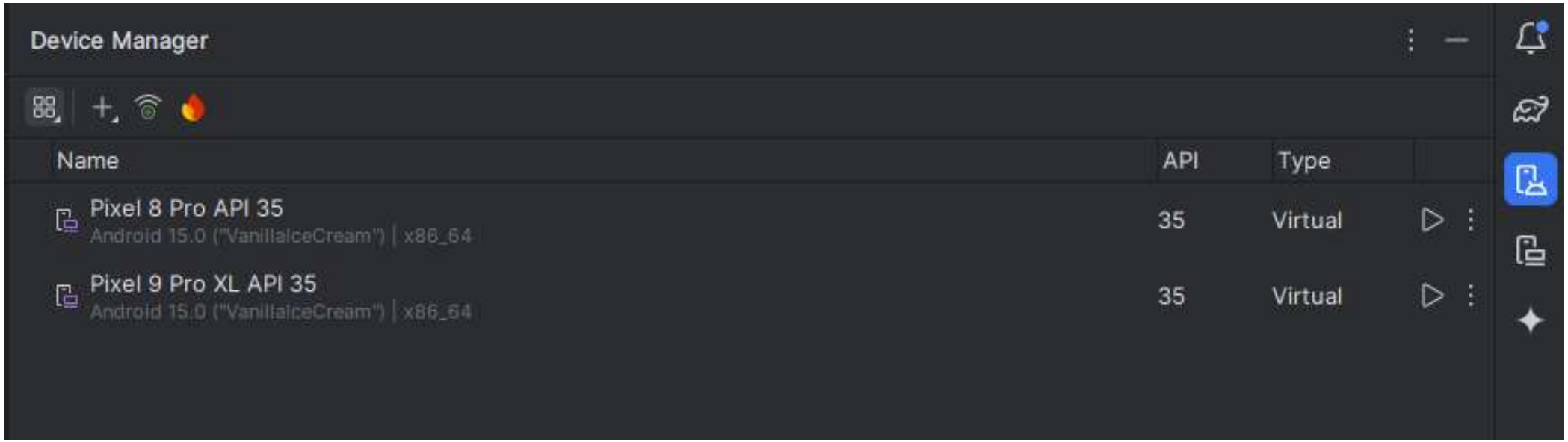
Criando um dispositivo



Criando um dispositivo

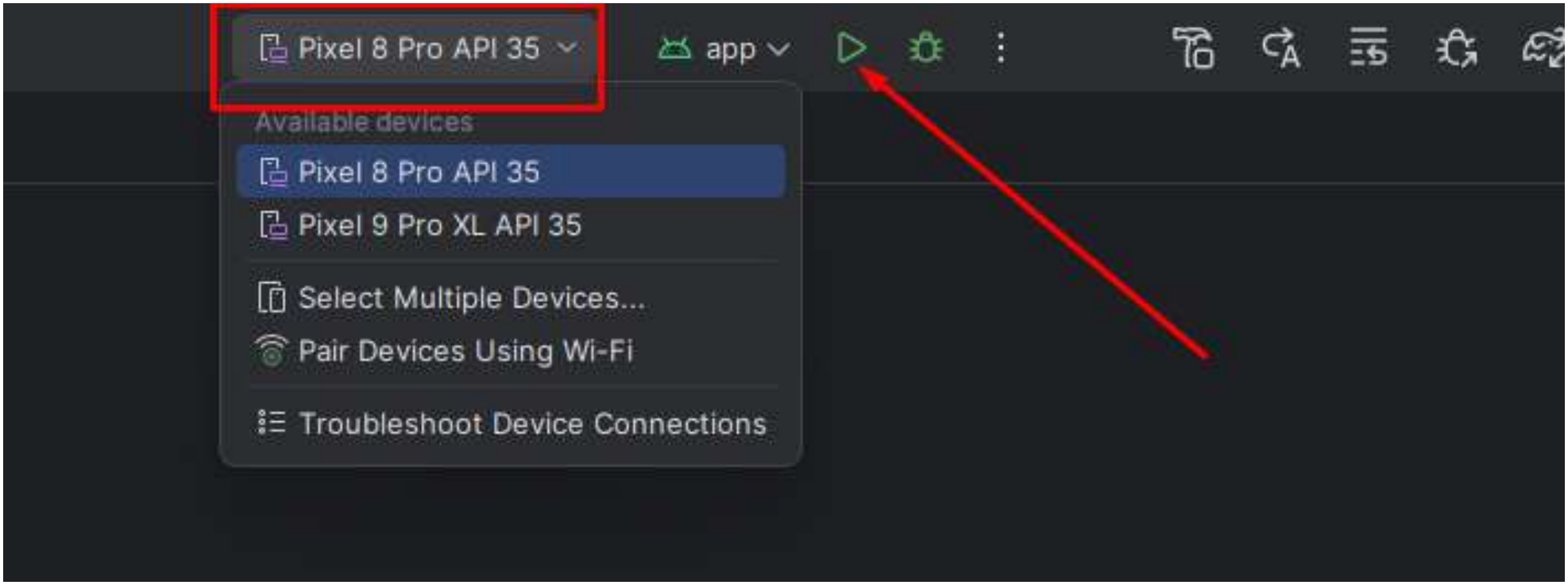


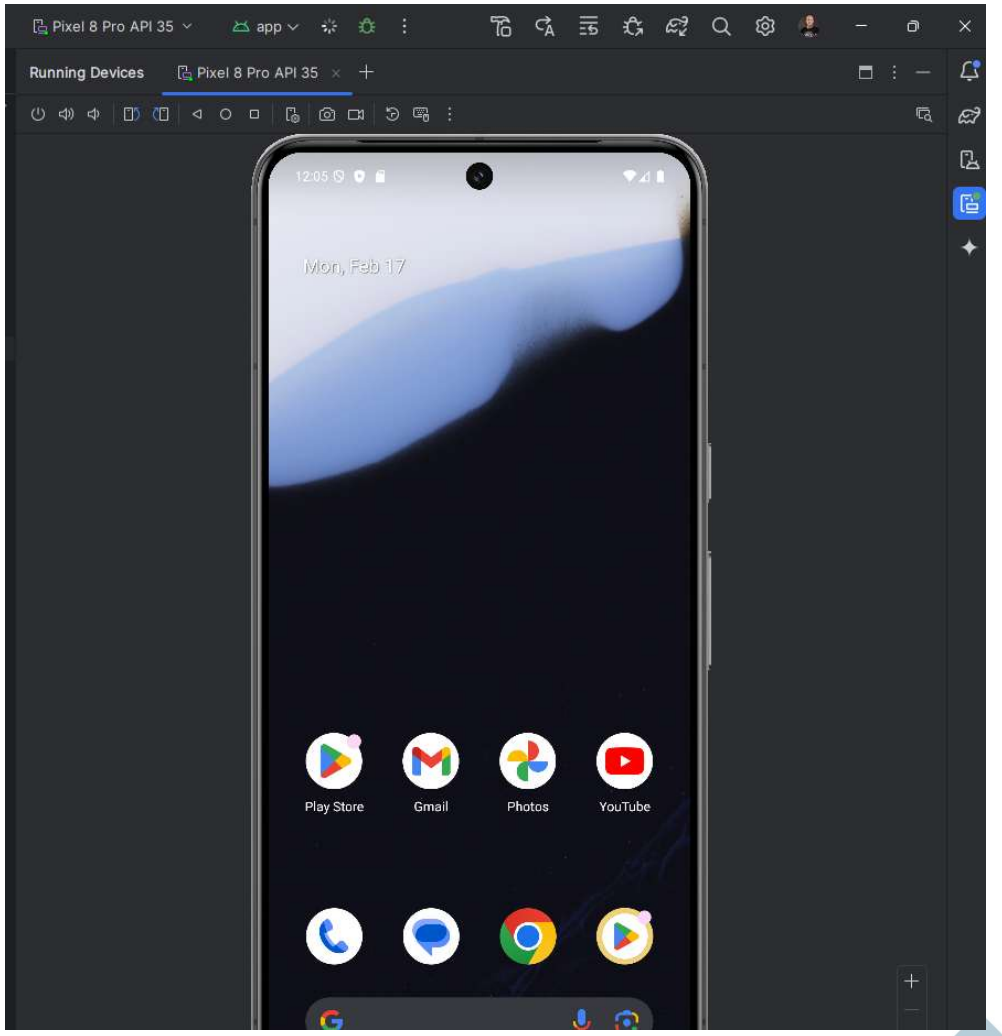
Dispositivos

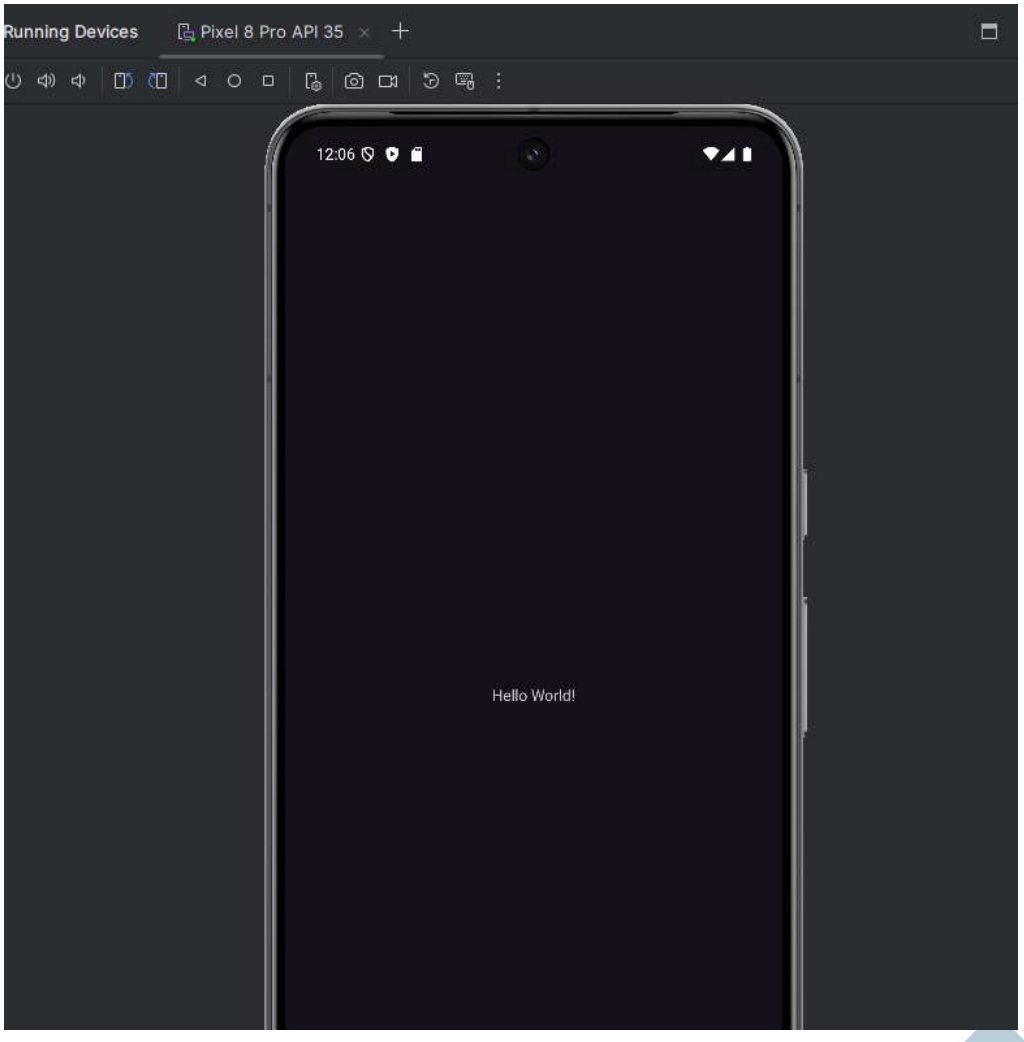


Rodando o nosso Hello World

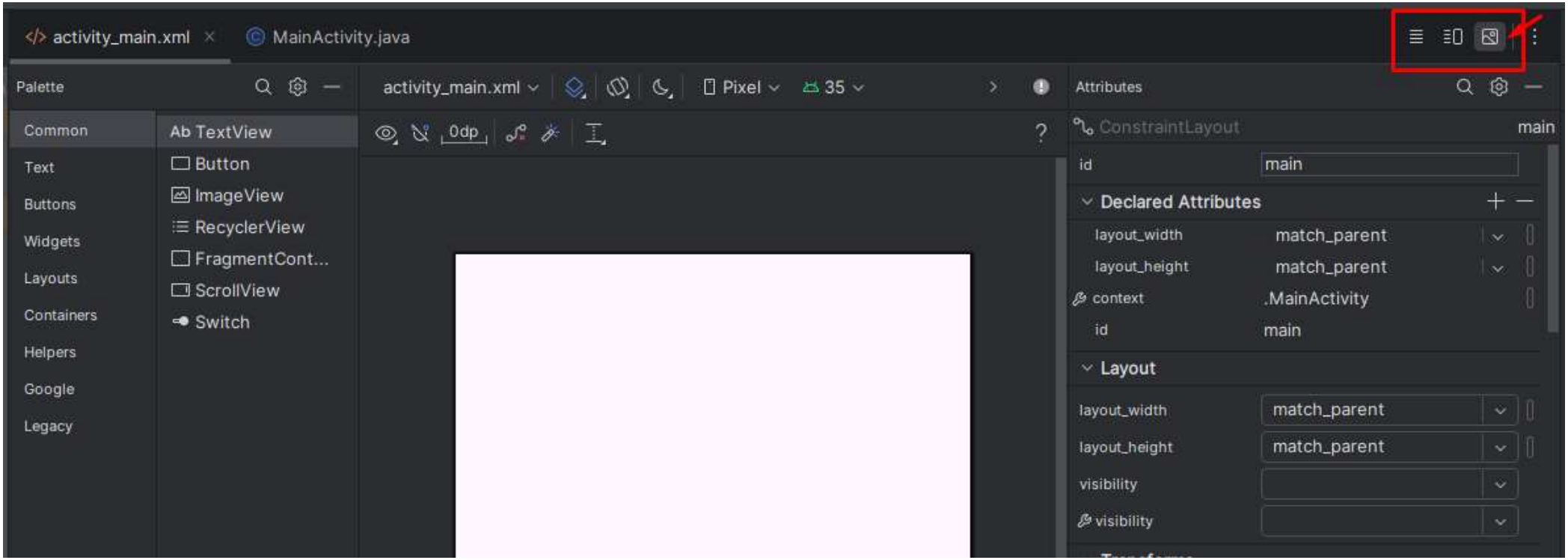
Selecione um emulador/dispositivo virtual e clique no Play



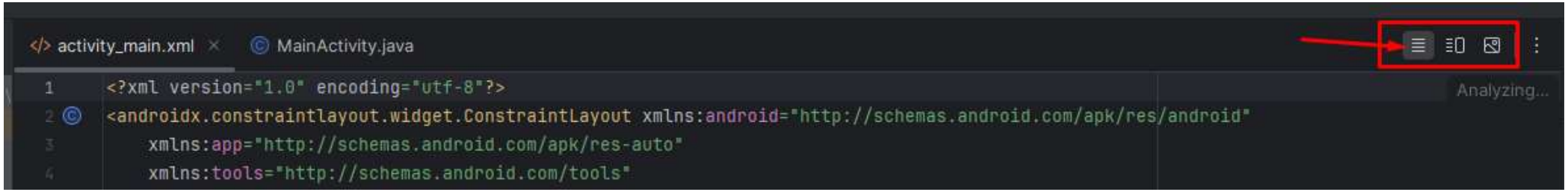

Pixel 8 Pro API 35
app
Running Devices
Pixel 8 Pro API 35
12:05
Mon, Feb 17
Play Store
Gmail
Photos
YouTube


Running Devices
Pixel 8 Pro API 35
12:06
Hello World!

activity_main.xml



activity_main.xml



activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.HelloWorld"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

MainActivity.Java

```java
package com.ernakh.helloworld;

import android.os.Bundle;

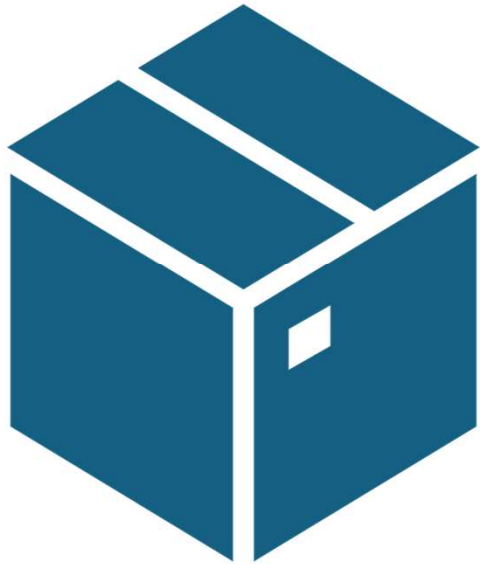
import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_main);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });
    }
}
```

MainActivity.Java

- package com.ernakh.helloworld;
- Define o nome do pacote do projeto. O pacote ajuda a organizar o código e evitar conflitos de nomes.



MainActivity.Java



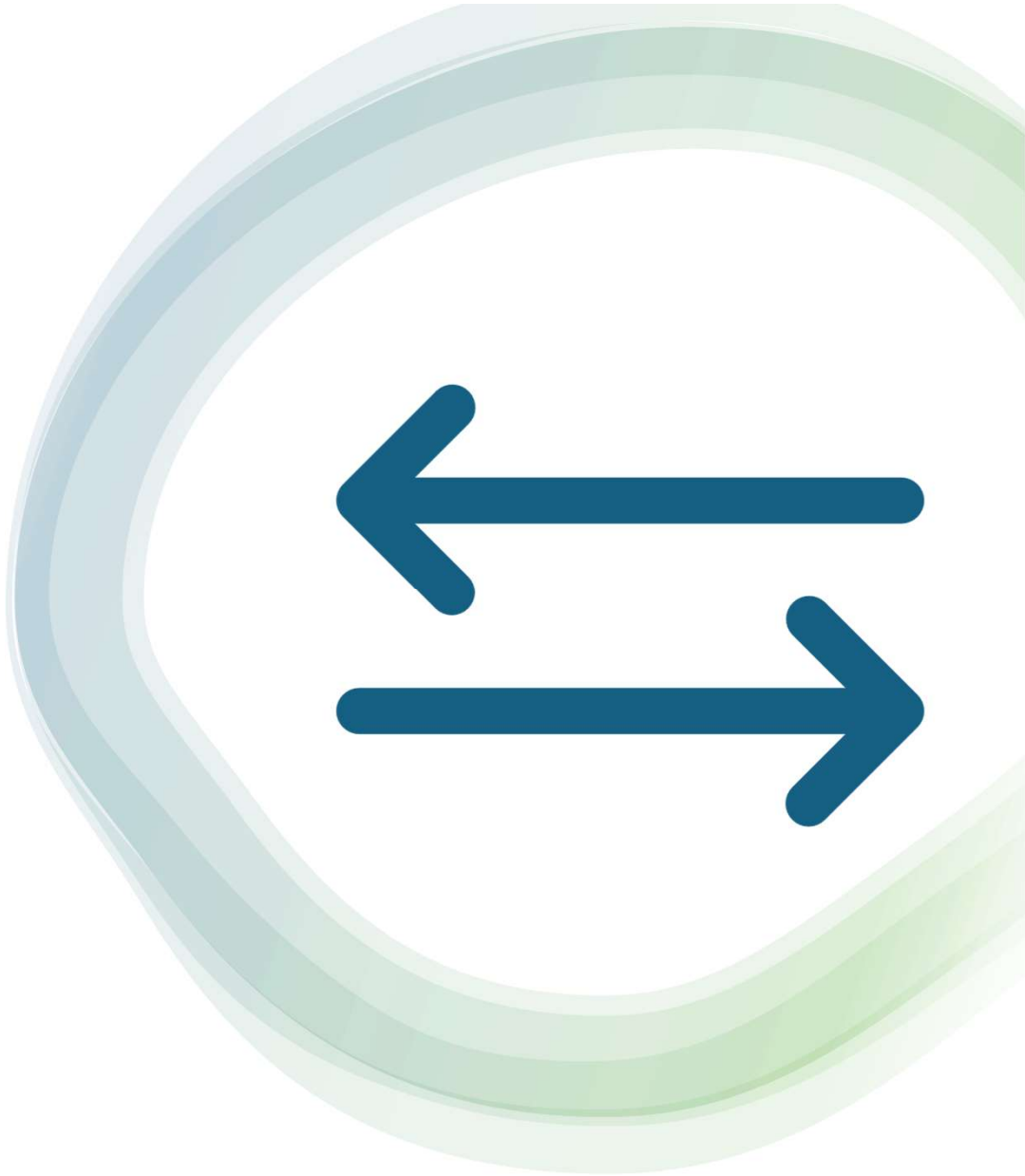
```
import android.os.Bundle;
```



Importa a classe **Bundle**, que é usada para armazenar dados entre estados de atividade (como quando a tela é girada).

MainActivity.Java

- import androidx.activity.EdgeToEdge;
- Importa EdgeToEdge, uma API usada para permitir que a interface ocupe toda a tela, incluindo áreas como a barra de status.





MainActivity.Java

- import androidx.appcompat.app.AppCompatActivity;
- Importa AppCompatActivity, que é uma classe base para atividades que usam a biblioteca de compatibilidade do Android. Permite compatibilidade com versões mais antigas.

MainActivity.Java

- import androidx.core.graphics.Insets;
- Importa a classe Insets, que representa espaços vazios, como a barra de status e a barra de navegação.

MainActivity.Java

- import androidx.core.view.ViewCompat;
- Importa ViewCompat, que contém métodos de compatibilidade para manipular View de maneira segura em diferentes versões do Android.

MainActivity.Java

- import androidx.core.view.WindowInsetsCompat;
- Importa WindowInsetsCompat, que fornece uma maneira compatível de obter informações sobre os insets da janela (por exemplo, onde a interface deve evitar sobreposição com elementos do sistema, como a barra de navegação).

MainActivity.Java

```
public class MainActivity
extends
AppCompatActivity {
```

Define MainActivity, que herda de AppCompatActivity, tornando-se uma atividade do Android com suporte a recursos modernos de UI.

MainActivity.Java

- @override
- protected void onCreate(Bundle savedInstanceState) {

- Define o método onCreate, que é chamado quando a atividade é criada. Ele recebe um Bundle com o estado salvo da atividade.

MainActivity.Java

- super.onCreate(savedInstanceState);
- Chama o método onCreate da classe pai (AppCompatActivity) para garantir que a inicialização básica da atividade ocorra corretamente.

- EdgeToEdge.enable(this);
- Ativa o modo "Edge to Edge", que permite que a interface ocupe toda a tela, inclusive as áreas sob a barra de status e a barra de navegação.javaCopyEdit

MainActivity.Java

- setContentView(R.layout.activity_main);
- Define o layout da atividade, carregando o arquivo XML activity_main.xml, que contém a interface gráfica.

MainActivity.Java

- ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -
  > {
- Adiciona um listener (setOnApplyWindowInsetsListener) à View com ID main, permitindo modificar os insets da janela quando forem aplicados.



Insets

- No desenvolvimento Android, **Insets** representam **as áreas ocupadas por elementos do sistema**, como **status bar, navigation bar, gestos de navegação e notch** (entalhe da tela).
- Eles ajudam os desenvolvedores a **ajustar automaticamente os layouts** para evitar sobreposição com esses elementos.





Insets

- Em dispositivos modernos, a tela pode ter **bordas curvas, entalhes (notch), botões virtuais e gestos de navegação**, que podem interferir no design do app.
- O sistema fornece os **insets** para que os desenvolvedores consigam adaptar a interface corretamente.
- **Exemplo de problemas sem Insets:**
  - Um botão pode ficar escondido atrás da **navigation bar**.
  - Um título pode ser cortado pela **status bar**.
  - Uma imagem pode ser sobreposta pelo **notch**.

MainActivity.Java

- Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
- Obtém os insets do sistema (como barra de status e barra de navegação) e os armazena em systemBars.

MainActivity.Java

- v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
- Define um padding para a View, garantindo que o conteúdo não fique coberto por barras do sistema.

Intents

Intents são **mensagens** usadas para iniciar atividades, serviços ou transmitir informações entre componentes.
✓ **Intents Explícitos** → Chamam **atividades específicas** dentro do app.
✓ **Intents Implícitos** → Pedem ao Android que escolha o app mais adequado para a ação.
✓ **Filtros de Intent (**`<intent-filter>`) permitem que o app **responda a ações do sistema** ou de outros apps.

Eles são fundamentais para a comunicação entre apps e componentes no Android!

Referências

- CALVO, Andres. Beginning Android wearables. Nova York: Apress, 2015.
- DEITEL, Paul; DEITEL, Harvey; DEITEL, Abbey. Android: como programar. 2. ed. Porto Alegre: Bookman, 2015.
- DEITEL, Paul J. et al. Android for programmers: an app-driven approach. Upper Saddle River, NJ: Prentice-Hall, 2012.
- LECHETA, Ricardo R. Google Android: aprenda a criar aplicações para dispositivos móveis com o Android SDK. 3. ed. São Paulo: Novatec, 2010.
- LECHETA, Ricardo R. Google Android para tablets: aprenda a criar aplicações para dispositivos móveis com o Android - de smartphones a tablets. São Paulo: Novatec, 2012
- Londero, F. T., Perozzo, R. F., Cantarelli, G. S., & Kurtz, G. C. (2015). Opção de marketing para sistemas Android por meio de geolocalização. Disciplinarum Scientia| Naturais e Tecnológicas, 16(1), 109-130.